



Getting Data

CPSC 4300/6300: Applied Data Science

Dr. Tim Ransom

Clemson University

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

Table of Contents



Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

2 Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data
└─ Data Acquisition

└─ Table of Contents

Table of Contents

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

Where does data live?



- **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).

3 Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

2026-01-06

Getting Data

Data Acquisition

Where does data live?

Where does data live?

- **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).

Where does data live?



- ▶ **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).
- ▶ **Databases:** Organized collections of data, managed by a Database Management System (DBMS).

3 Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

2026-01-06

Getting Data Data Acquisition

Where does data live?

Where does data live?

- ▶ **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).
- ▶ **Databases:** Organized collections of data, managed by a Database Management System (DBMS).

Where does data live?



- ▶ **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).
- ▶ **Databases:** Organized collections of data, managed by a Database Management System (DBMS).
- ▶ **Web:** APIs (Application Programming Interfaces), web scraping.

3 Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

2026-01-06

Getting Data └─ Data Acquisition

└─ Where does data live?

Where does data live?

- ▶ **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).
- ▶ **Databases:** Organized collections of data, managed by a Database Management System (DBMS).
- ▶ **Web:** APIs (Application Programming Interfaces), web scraping.

Where does data live?



3 Data Acquisition

- ▶ **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).
- ▶ **Databases:** Organized collections of data, managed by a Database Management System (DBMS).
- ▶ **Web:** APIs (Application Programming Interfaces), web scraping.
- ▶ **Data Warehouses/Lakes:** Centralized repositories for large volumes of data from various sources.

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data └─ Data Acquisition

└─ Where does data live?

Where does data live?

- ▶ **Files:** Simple text files (CSV, TSV), structured formats (JSON, XML, Parquet), spreadsheets (Excel).
- ▶ **Databases:** Organized collections of data, managed by a Database Management System (DBMS).
- ▶ **Web:** APIs (Application Programming Interfaces), web scraping.
- ▶ **Data Warehouses/Lakes:** Centralized repositories for large volumes of data from various sources.

Data in files (CSV, JSON, Excel)



► CSV (Comma-Separated Values):

- Plain text, tabular data. Each row is a line, columns are separated by commas.
- Simple, human-readable, widely supported.

4 Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data

└─ Data Acquisition

└─ Data in files (CSV, JSON, Excel)

Data in files (CSV, JSON, Excel)

- CSV (Comma-Separated Values):
 - Plain text, tabular data. Each row is a line, columns are separated by commas.
 - Simple, human-readable, widely supported.

Data in files (CSV, JSON, Excel)



4 Data Acquisition

- ▶ **CSV (Comma-Separated Values):**
 - ▶ Plain text, tabular data. Each row is a line, columns are separated by commas.
 - ▶ Simple, human-readable, widely supported.
- ▶ **JSON (JavaScript Object Notation):**
 - ▶ Text format for semi-structured data. Uses key-value pairs and nested structures.
 - ▶ Common for web APIs, flexible schema.

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data └ Data Acquisition

└ Data in files (CSV, JSON, Excel)

Data in files (CSV, JSON, Excel)

- ▶ **CSV (Comma-Separated Values):**
 - ▶ Plain text, tabular data. Each row is a line, columns are separated by commas.
 - ▶ Simple, human-readable, widely supported.
- ▶ **JSON (JavaScript Object Notation):**
 - ▶ Text format for semi-structured data. Uses key-value pairs and nested structures.
 - ▶ Common for web APIs, flexible schema.

Data in files (CSV, JSON, Excel)



4 Data Acquisition

- ▶ **CSV (Comma-Separated Values):**
 - ▶ Plain text, tabular data. Each row is a line, columns are separated by commas.
 - ▶ Simple, human-readable, widely supported.
- ▶ **JSON (JavaScript Object Notation):**
 - ▶ Text format for semi-structured data. Uses key-value pairs and nested structures.
 - ▶ Common for web APIs, flexible schema.
- ▶ **Excel (XLSX):**
 - ▶ Proprietary spreadsheet format. Can contain multiple sheets, formulas, formatting.
 - ▶ Good for manual data entry and analysis, but less ideal for programmatic access.

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data └ Data Acquisition

└ Data in files (CSV, JSON, Excel)

Data in files (CSV, JSON, Excel)

- ▶ **CSV (Comma-Separated Values):**
 - ▶ Plain text, tabular data. Each row is a line, columns are separated by commas.
 - ▶ Simple, human-readable, widely supported.
- ▶ **JSON (JavaScript Object Notation):**
 - ▶ Text format for semi-structured data. Uses key-value pairs and nested structures.
 - ▶ Common for web APIs, flexible schema.
- ▶ **Excel (XLSX):**
 - ▶ Proprietary spreadsheet format. Can contain multiple sheets, formulas, formatting.
 - ▶ Good for manual data entry and analysis, but less ideal for programmatic access.

Data in databases (Relational, NoSQL)



5 Data Acquisition

► Relational Databases (SQL):

- Organize data into tables with predefined schemas (rows and columns).
- Enforce relationships between tables using keys.
- Examples: PostgreSQL, MySQL, SQLite.

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Data Acquisition

└─ Data in databases (Relational, NoSQL)

Data in databases (Relational, NoSQL)

- Relational Databases (SQL):
 - Organize data into tables with predefined schemas (rows and columns).
 - Enforce relationships between tables using keys.
 - Examples: PostgreSQL, MySQL, SQLite.

Data in databases (Relational, NoSQL)



5 Data Acquisition

► Relational Databases (SQL):

- Organize data into tables with predefined schemas (rows and columns).
- Enforce relationships between tables using keys.
- Examples: PostgreSQL, MySQL, SQLite.

► NoSQL Databases:

- Do not use the traditional table-based relational model.
- Better for unstructured or semi-structured data, high scalability.
- Types: Document (MongoDB), Key-Value (Redis), Column-family (Cassandra), Graph (Neo4j).

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data

└─ Data Acquisition

└─ Data in databases (Relational, NoSQL)

Data in databases (Relational, NoSQL)

- **Relational Databases (SQL):**
 - Organize data into tables with predefined schemas (rows and columns).
 - Enforce relationships between tables using keys.
 - Examples: PostgreSQL, MySQL, SQLite.
- **NoSQL Databases:**
 - Do not use the traditional table-based relational model.
 - Better for unstructured or semi-structured data, high scalability.
 - Types: Document (MongoDB), Key-Value (Redis), Column-family (Cassandra), Graph (Neo4j).

Data from the web (APIs, Scraping)



6 Data Acquisition

► APIs (Application Programming Interfaces):

- A structured way for applications to communicate with each other.
- The data provider defines an endpoint; you send a request and get data back (often in JSON).
- **The preferred method:** reliable, stable, and legal.

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data

└ Data Acquisition

└ Data from the web (APIs, Scraping)

Data from the web (APIs, Scraping)

- **APIs (Application Programming Interfaces):**
 - A structured way for applications to communicate with each other.
 - The data provider defines an endpoint; you send a request and get data back (often in JSON).
 - **The preferred method:** reliable, stable, and legal.

Data from the web (APIs, Scraping)



6 Data Acquisition

▶ APIs (Application Programming Interfaces):

- ▶ A structured way for applications to communicate with each other.
- ▶ The data provider defines an endpoint; you send a request and get data back (often in JSON).
- ▶ **The preferred method:** reliable, stable, and legal.

▶ Web Scraping:

- ▶ Extracting data directly from the HTML of websites.
- ▶ Used when no API is available.
- ▶ Can be brittle (breaks if the website layout changes) and may have legal/ethical implications.

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

2026-01-06

Getting Data

└ Data Acquisition

└ Data from the web (APIs, Scraping)

Data from the web (APIs, Scraping)

- ▶ **APIs (Application Programming Interfaces):**
 - ▶ A structured way for applications to communicate with each other.
 - ▶ The data provider defines an endpoint; you send a request and get data back (often in JSON).
 - ▶ **The preferred method:** reliable, stable, and legal.
- ▶ **Web Scraping:**
 - ▶ Extracting data directly from the HTML of websites.
 - ▶ Used when no API is available.
 - ▶ Can be brittle (breaks if the website layout changes) and may have legal/ethical implications.

Data Warehouses vs. Transactional Databases (OLTP vs. OLAP)



Feature	Transactional Database (OLTP)	Data Warehouse (OLAP)
Primary Goal	Run the business (day-to-day)	Analyze the business
Workload	Many small, fast transactions	Few complex, long queries
Data	Real-time, current data	Historical, aggregated data
Schema	Normalized (less redundancy)	Denormalized (for performance)
Example Use	E-commerce order entry	Quarterly sales analysis
Stands for...	Online Transaction Processing	Online Analytical Processing

7 Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

2026-01-06

Getting Data
Data Acquisition

Data Warehouses vs. Transactional Databases (OLTP vs. OLAP)

Data Warehouses vs. Transactional Databases (OLTP vs. OLAP)

Feature	Transactional Database (OLTP)	Data Warehouse (OLAP)
Primary Goal	Run the business (day-to-day)	Analyze the business
Workload	Many small, fast transactions	Few complex, long queries
Data	Real-time, current data	Historical, aggregated data
Schema	Normalized (less redundancy)	Denormalized (for performance)
Example Use	E-commerce order entry	Quarterly sales analysis
Stands for...	Online Transaction Processing	Online Analytical Processing

Table of Contents



- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

8

- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

57

2026-01-06

- Getting Data
 - Acquiring Data from SQL
 - Table of Contents

- Table of Contents
- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

What is a Relational Database?



A relational database organizes data into a set of tables. Each table has a schema defining its columns and data types. Tables are related to one another through keys.

Data Acquisition

9 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from SQL

└─ What is a Relational Database?

What is a Relational Database?

A relational database organizes data into a set of tables. Each table has a schema defining its columns and data types. Tables are related to one another through keys.

Tables, Rows, Columns



- ▶ **Table (Relation):** A collection of related data (e.g., Customers, Products).
- ▶ **Column (Attribute):** A named data field with a specific type (e.g., customer_name, price).
- ▶ **Row (Tuple):** A single record in a table (e.g., one specific customer).

```
-- Example: A 'Customers' table
CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  email VARCHAR(100)
);
```

Data Acquisition

10 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

Acquiring Data from SQL

Tables, Rows, Columns

Tables, Rows, Columns

- ▶ **Table (Relation):** A collection of related data (e.g., Customers, Products).
- ▶ **Column (Attribute):** A named data field with a specific type (e.g., customer_name, price).
- ▶ **Row (Tuple):** A single record in a table (e.g., one specific customer).

```
-- Example: A 'Customers' table
CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  email VARCHAR(100)
);
```

Primary Keys & Foreign Keys



- **Primary Key (PK):** A column (or set of columns) that **uniquely identifies** each row in a table. Cannot be null.

Data Acquisition

11 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from SQL

└─ Primary Keys & Foreign Keys

Primary Keys & Foreign Keys

► **Primary Key (PK):** A column (or set of columns) that **uniquely identifies** each row in a table. Cannot be null.

Primary Keys & Foreign Keys



- ▶ **Primary Key (PK):** A column (or set of columns) that **uniquely identifies** each row in a table. Cannot be null.
- ▶ **Foreign Key (FK):** A column in one table that is a **reference** to the primary key of another table. It establishes a link between the two tables.

Data Acquisition

11 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data
└─ Acquiring Data from SQL

└─ Primary Keys & Foreign Keys

Primary Keys & Foreign Keys

- ▶ **Primary Key (PK):** A column (or set of columns) that **uniquely identifies** each row in a table. Cannot be null.
- ▶ **Foreign Key (FK):** A column in one table that is a **reference** to the primary key of another table. It establishes a link between the two tables.

The SQL SELECT Statement



Used to query the database and retrieve data that matches criteria that you specify. Be aware that SQL can do much much more advanced and useful things, but we're just going to use it to get the data, not compute the data.

Data Acquisition

12 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from SQL

└─ The SQL SELECT Statement

The SQL SELECT Statement

Used to query the database and retrieve data that matches criteria that you specify. Be aware that SQL can do much much more advanced and useful things, but we're just going to use it to get the data, not compute the data.

SELECT col_1, col_2



Specifies the columns you want to retrieve. Use * to select all columns.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT username, email FROM users;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

Data Acquisition

13 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

Acquiring Data from SQL

SELECT col_1, col_2

SELECT col_1, col_2

Specifies the columns you want to retrieve. Use * to select all columns.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT username, email FROM users;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

FROM my_table

Specifies the table to retrieve the data from.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM users;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```



Data Acquisition

14 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data
└─ Acquiring Data from SQL

└─ FROM my_table

FROM my_table

Specifies the table to retrieve the data from.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM users;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

WHERE some_condition



Filters records based on a specific condition. Important for speed.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM users WHERE id = 1;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

Data Acquisition

15 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from SQL

└─ WHERE some_condition

WHERE some_condition

Filters records based on a specific condition. Important for speed.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM users WHERE id = 1;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

LIMIT n

Restricts the number of rows returned by the query.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM users LIMIT 2;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```



Data Acquisition

16 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data
└─ Acquiring Data from SQL

└─ LIMIT n

LIMIT n

Restricts the number of rows returned by the query.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM users LIMIT 2;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```


Operators: =, <, >, !=, LIKE, IN



Data Acquisition

17 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

- Use standard operators to compare values.
- LIKE is used for pattern matching in strings (e.g., `WHERE name LIKE 'A%'`).
- IN checks if a value is within a list of values (e.g., `WHERE country IN ('USA', 'Canada')`).

2026-01-06

Getting Data

Acquiring Data from SQL

Operators: =, <, >, !=, LIKE, IN

Operators: =, <, >, !=, LIKE, IN

- Use standard operators to compare values.
- LIKE is used for pattern matching in strings (e.g., `WHERE name LIKE 'A%'`).
- IN checks if a value is within a list of values (e.g., `WHERE country IN ('USA', 'Canada')`).

Combining: AND, OR, NOT



Combine multiple conditions in your WHERE clause.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM tasks WHERE status = 'pending' AND user_id = 3;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

Data Acquisition

18 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from SQL

└─ Combining: AND, OR, NOT

Combining: AND, OR, NOT

Combine multiple conditions in your WHERE clause.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM tasks WHERE status = 'pending' AND user_id = 3;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

ORDER BY col_1 [ASC|DESC]

Sorts the result set by one or more columns. Default is ASC (ascending).

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM tasks ORDER BY title DESC;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```



Data Acquisition

19 Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

Acquiring Data from SQL

ORDER BY col_1 [ASC|DESC]

ORDER BY col_1 [ASC|DESC]

Sorts the result set by one or more columns. Default is ASC (ascending).

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('lectures/data/test.db')
query = "SELECT * FROM tasks ORDER BY title DESC;"
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

Table of Contents



- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

- Data Acquisition
- Acquiring Data from SQL
- 20 Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)
- 57

2026-01-06

- Getting Data
 - Combining Data from Multiple Tables
- Table of Contents

Table of Contents

- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

What are JOINS?



JOIN clauses are used to combine rows from two or more tables based on a related column between them.

- Data Acquisition
 - Acquiring Data from SQL
 - 21 Combining Data from Multiple Tables**
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)
- 57**

2026-01-06

- Getting Data
 - Combining Data from Multiple Tables
 - What are JOINS?

What are JOINS?

JOIN clauses are used to combine rows from two or more tables based on a related column between them.

FULL OUTER JOIN

Returns all records when there is a match in either left or right table. Returns NULLs for non-matching columns.



Data Acquisition

Acquiring Data from SQL

22 Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

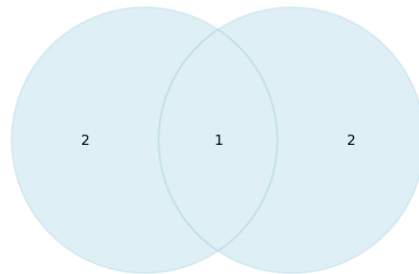
Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

FULL OUTER JOIN



Left Table

Right Table

```
pd.merge(left, right, \
        on='key', \
        how='outer')
```

2026-01-06

Getting Data

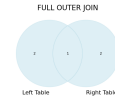
Combining Data from Multiple Tables

FULL OUTER JOIN

FULL OUTER JOIN

Returns all records when there is a match in either left or right table. Returns NULLs for non-matching columns.

```
pd.merge(left, right, \
        on='key', \
        how='outer')
```



INNER JOIN



Returns records that have matching values in both tables.

INNER JOIN



Left Table

Right Table

```
pd.merge(left, right, \
         on='key', \
         how='inner')
```

Data Acquisition

Acquiring Data
from SQL

23 Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

Combining Data from Multiple Tables

INNER JOIN

INNER JOIN

Returns records that have matching values in both tables.

```
pd.merge(left, right, \
         on='key', \
         how='inner')
```

INNER JOIN



Left Table Right Table

LEFT JOIN

Returns all records from the left table, and the matched records from the right table. The result is NULL from the right side if there is no match.



Data Acquisition

Acquiring Data from SQL

24 Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

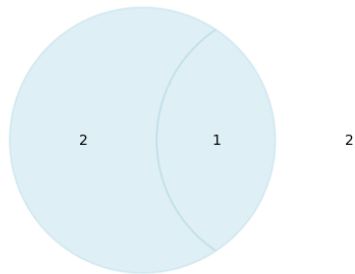
Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

LEFT JOIN



Left Table

Right Table

2026-01-06

Getting Data

Combining Data from Multiple Tables

LEFT JOIN

LEFT JOIN

Returns all records from the left table, and the matched records from the right table. The result is NULL from the right side if there is no match.

```
pd.merge(left, right, \n         on='key', \n         how='left')
```



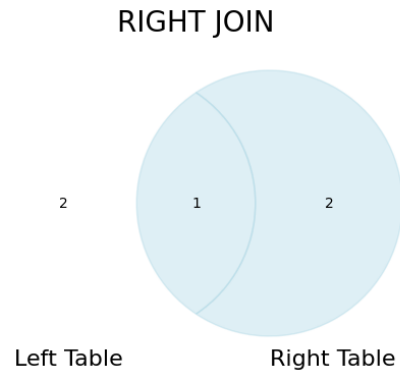
Left Table

Right Table

RIGHT JOIN

Returns all records from the right table, and the matched records from the left table. The result is NULL from the left side when there is no match.

```
pd.merge(left, right, \
        on='key', \
        how='right')
```



Data Acquisition

Acquiring Data
from SQL

25 Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

Combining Data from Multiple Tables

RIGHT JOIN

RIGHT JOIN

Returns all records from the right table, and the matched records from the left table. The result is NULL from the left side when there is no match.

```
pd.merge(left, right, \
        on='key', \
        how='right')
```



Example: Joining customer and order tables (1/4)



List the users and tasks to show what is going to be joined together

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('data/test.db')
print("--- Users Table ---")
users_df = pd.read_sql_query("SELECT * FROM users;", conn)
print(users_df)
print("\n--- Tasks Table ---")
tasks_df = pd.read_sql_query("SELECT * FROM tasks;", conn)
print(tasks_df)
conn.close()
```

Data Acquisition

Acquiring Data
from SQL

26 Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

Combining Data from Multiple Tables

Example: Joining customer and order tables (1/4)

Example: Joining customer and order tables (1/4)

List the users and tasks to show what is going to be joined together

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('data/test.db')
print("--- Users Table ---")
users_df = pd.read_sql_query("SELECT * FROM users;", conn)
print(users_df)
print("\n--- Tasks Table ---")
tasks_df = pd.read_sql_query("SELECT * FROM tasks;", conn)
print(tasks_df)
conn.close()
```

Example: Joining customer and order tables (2/4)



--- Users Table ---					
	id	username	email	created_at	
0	1	alice_w	alice@example.com	2025-11-08	21:36:56
1	2	bob_builder	bob@constructs.com	2025-11-08	21:36:56
2	3	charlie_dev	charlie@code.net	2025-11-08	21:36:56

--- Tasks Table ---				
	id	user_id	title	status
0	1	1	Fix login bug	completed
1	2	1	Write documentation	pending
2	3	2	Build new dashboard	in-progress
3	4	3	Set up CI/CD pipeline	pending
4	5	3	Refactor database schema	pending

Data Acquisition

Acquiring Data from SQL

27 Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

2026-01-06

Getting Data Combining Data from Multiple Tables

Example: Joining customer and order tables (2/4)

Example: Joining customer and order tables (2/4)

--- Users Table ---				
	id	username	email	created_at
0	1	alice_w	alice@example.com	2025-11-08 21:36:56
1	2	bob_builder	bob@constructs.com	2025-11-08 21:36:56
2	3	charlie_dev	charlie@code.net	2025-11-08 21:36:56

--- Tasks Table ---				
	id	user_id	title	status
0	1	1	Fix login bug	completed
1	2	1	Write documentation	pending
2	3	2	Build new dashboard	in-progress
3	4	3	Set up CI/CD pipeline	pending
4	5	3	Refactor database schema	pending

Example: Joining customer and order tables (3/4)

Find the name of each user along with their assigned tasks.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('data/test.db')
query = """
SELECT u.username, t.title
FROM users u
INNER JOIN
    tasks t ON u.id = t.user_id;
"""
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```



Data Acquisition

Acquiring Data
from SQL

28

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

Combining Data from Multiple Tables

Example: Joining customer and order tables (3/4)

Example: Joining customer and order tables (3/4)

Find the name of each user along with their assigned tasks.

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('data/test.db')
query = """
SELECT u.username, t.title
FROM users u
INNER JOIN
    tasks t ON u.id = t.user_id;
"""
df = pd.read_sql_query(query, conn)
conn.close()
print(df)
```

Example: Joining customer and order tables (4/4)



	username	title
0	alice_w	Fix login bug
1	alice_w	Write documentation
2	bob_builder	Build new dashboard
3	charlie_dev	Set up CI/CD pipeline
4	charlie_dev	Refactor database schema

Data Acquisition

Acquiring Data from SQL

29 Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

Combining Data from Multiple Tables

Example: Joining customer and order tables (4/4)

Example: Joining customer and order tables (4/4)

	username	title
0	alice_w	Fix login bug
1	alice_w	Write documentation
2	bob_builder	Build new dashboard
3	charlie_dev	Set up CI/CD pipeline
4	charlie_dev	Refactor database schema

Table of Contents



- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

30

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

- Getting Data
 - Acquiring Data from Web APIs
- Table of Contents

- Table of Contents
- Data Acquisition
- Acquiring Data from SQL
- Combining Data from Multiple Tables
- Acquiring Data from Web APIs
- Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

What is an API?



An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other. For data acquisition, we typically use web APIs to request data from a server.

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

31 Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from Web APIs

└─ What is an API?

What is an API?

An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other. For data acquisition, we typically use web APIs to request data from a server.

Understanding REST APIs

REST (REpresentational State Transfer) is a popular architectural style for web APIs.

Request methods: GET, POST

- **GET**: Requests data from a specified resource. This is the most common method for data acquisition.



Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

32 Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from Web APIs

└─ Understanding REST APIs

Understanding REST APIs

REST (REpresentational State Transfer) is a popular architectural style for web APIs.

Request methods: GET, POST

► **GET**: Requests data from a specified resource. This is the most common method for data acquisition.

Understanding REST APIs

REST (REpresentational State Transfer) is a popular architectural style for web APIs.

Request methods: GET, POST

- ▶ **GET:** Requests data from a specified resource. This is the most common method for data acquisition.
- ▶ **POST:** Submits data to be processed to a specified resource (e.g., creating a new user).

API Endpoints

An endpoint is a specific URL where an API can be accessed. For example, `https://api.example.com/v1/users` could be an endpoint to get a list of users.



Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

32 Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

Acquiring Data from Web APIs

Understanding REST APIs

Understanding REST APIs

REST (REpresentational State Transfer) is a popular architectural style for web APIs.

Request methods: GET, POST

▶ **GET:** Requests data from a specified resource. This is the most common method for data acquisition.

▶ **POST:** Submits data to be processed to a specified resource (e.g., creating a new user).

API Endpoints

An endpoint is a specific URL where an API can be accessed. For example, `https://api.example.com/v1/users` could be an endpoint to get a list of users.

Working with JSON Data



Structure: Keys and Values

JSON (JavaScript Object Notation) data is composed of key-value pairs, similar to Python dictionaries. It supports strings, numbers, booleans, arrays (lists), and nested objects (dictionaries).

Parsing JSON in Python (the requests and json libraries)

The requests library can automatically decode JSON responses into a Python dictionary. The json library can be used for more manual parsing if needed.

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

33 Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from Web APIs

└─ Working with JSON Data

Working with JSON Data

Structure: Keys and Values

JSON (JavaScript Object Notation) data is composed of key-value pairs, similar to Python dictionaries. It supports strings, numbers, booleans, arrays (lists), and nested objects (dictionaries).

Parsing JSON in Python (the requests and json libraries)

The requests library can automatically decode JSON responses into a Python dictionary. The json library can be used for more manual parsing if needed.

Example of what JSON looks like

```
{  
  "firstName": "Jane", // string  
  "age": 42, // integer  
  "isStudent": false, //boolean  
  "skills": [ // list of strings  
    "Python",  
    "Data Analysis",  
    "Public Speaking"  
  ],  
  "address": { // hashmap of strings  
    "street": "123 Main St",  
    "city": "Anytown",  
    "zipCode": "12345"  
  }  
}
```



Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

34 Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Acquiring Data from Web APIs

└─ Example of what JSON looks like

Example of what JSON looks like

```
{  
  "firstName": "Jane", // string  
  "age": 42, // integer  
  "isStudent": false, //boolean  
  "skills": [ // list of strings  
    "Python",  
    "Data Analysis",  
    "Public Speaking"  
  ],  
  "address": { // hashmap of strings  
    "street": "123 Main St",  
    "city": "Anytown",  
    "zipCode": "12345"  
  }  
}
```

Considerations of using JSON



JSON data is very rarely ready to be put into tables, it's regularly jagged and has mixed data types all over the place. It takes work to figure out how to tabularize data!
That's why data scientists prefer SQL to NoSQL!

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

35 Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data
└─ Acquiring Data from Web APIs

└─ Considerations of using JSON

Considerations of using JSON

JSON data is very rarely ready to be put into tables, it's regularly jagged and has mixed data types all over the place. It takes work to figure out how to tabularize data!
That's why data scientists prefer SQL to NoSQL!

Table of Contents



- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API**
 - Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)

- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - 36 Example: Calling a public API**
 - Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)

- Getting Data
 - Example: Calling a public API**
 - Table of Contents

- Table of Contents
 - Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API**
 - Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)

Making a GET request (curl)



```
curl -L \  
  -H "Accept: application/vnd.github+json" \  
  -H "X-GitHub-API-Version: 2022-11-28" \  
  "https://api.github.com/users/octocat" | head --lines=3
```

```
{  
  "login": "octocat",  
  "id": 583231,...
```

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

37 Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Example: Calling a public API

└─ Making a GET request (curl)

Making a GET request (curl)

```
curl -L \  
  -H "Accept: application/vnd.github+json" \  
  -H "X-GitHub-API-Version: 2022-11-28" \  
  "https://api.github.com/users/octocat" | head --lines=3
```

```
{  
  "login": "octocat",  
  "id": 583231,...
```

Making a GET request (Python)

```
import requests
url = "https://api.github.com/users/octocat"
headers = {"Accept": "application/vnd.github+json",
           "X-GitHub-API-Version": "2022-11-28"}
response = requests.get(url, headers=headers)
if response.status_code == 200:
    data = response.json()
    print(f"Login: {data['login']}")
```

Login: octocat



Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

38 Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Example: Calling a public API

└─ Making a GET request (Python)

Making a GET request (Python)

```
import requests
url = "https://api.github.com/users/octocat"
headers = {"Accept": "application/vnd.github+json",
           "X-GitHub-API-Version": "2022-11-28"}
response = requests.get(url, headers=headers)
if response.status_code == 200:
    data = response.json()
    print(f"Login: {data['login']}")
```

Login: octocat

Checking the status code

A status code of 200 means the request was successful. Anybody know what a 404 means?

```
if response.status_code == 200:
    print("Request successful!")
    # Parse JSON response into a Python list of dicts
    data = response.json()
else:
    print(f"Request failed with status code: \
        {response.status_code}")
```



Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

39 Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└ Example: Calling a public API

└ Checking the status code

Checking the status code

A status code of 200 means the request was successful. Anybody know what a 404 means?

```
if response.status_code == 200:
    print("Request successful!")
    # Parse JSON response into a Python list of dicts
    data = response.json()
else:
    print(f"Request failed with status code: \
        {response.status_code}")
```


Converting the JSON response to a Pandas DataFrame



This one can get a bit complicated, but check out the `from_dict` and `json_normalize` functions from pandas:

```
>>> pd.DataFrame.from_dict(pd.json_normalize(data))
  login      id  ...      created_at      updated_at
0  octocat  583231  ...  2011-01-25T18:44:36Z  2025-11-22T12:23:13Z
[1 rows x 33 columns]
```

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

40 Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

Example: Calling a public API

Converting the JSON response to a Pandas DataFrame

Converting the JSON response to a Pandas DataFrame

This one can get a bit complicated, but check out the `from_dict` and `json_normalize` functions from pandas:

```
>>> pd.DataFrame.from_dict(pd.json_normalize(data))
  login      id  ...      created_at      updated_at
0  octocat  583231  ...  2011-01-25T18:44:36Z  2025-11-22T12:23:13Z
[1 rows x 33 columns]
```

API Authentication (Briefly)



Many APIs require authentication to control access and track usage.

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

41 Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└ Example: Calling a public API

└ API Authentication (Briefly)

API Authentication (Briefly)

Many APIs require authentication to control access and track usage.

API Authentication (Briefly)



Many APIs require authentication to control access and track usage.

- ▶ **API Keys:** A unique string passed along with your request to identify you.

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

41 Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└ Example: Calling a public API

└ API Authentication (Briefly)

API Authentication (Briefly)

Many APIs require authentication to control access and track usage.
▶ **API Keys:** A unique string passed along with your request to identify you.

API Authentication (Briefly)



Many APIs require authentication to control access and track usage.

- ▶ **API Keys:** A unique string passed along with your request to identify you.
- ▶ **OAuth:** A more complex but secure standard for authorization, where you get temporary access tokens instead of using a static key.

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

41 Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└ Example: Calling a public API

└ API Authentication (Briefly)

API Authentication (Briefly)

Many APIs require authentication to control access and track usage.

- ▶ **API Keys:** A unique string passed along with your request to identify you.
- ▶ **OAuth:** A more complex but secure standard for authorization, where you get temporary access tokens instead of using a static key.

API Keys



Typically, you include your API key in the request headers or as a URL parameter.

Example with a header

```
headers = {"Authorization": "Bearer YOUR_API_KEY"}  
response = requests.get(url, headers=headers)
```

Example with a parameter

```
params = {"api_key": "YOUR_API_KEY"}  
response = requests.get(url, params=params)
```

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

42 Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└ Example: Calling a public API

└ API Keys

API Keys

Typically, you include your API key in the request headers or as a URL parameter.

```
# Example with a header  
headers = {"Authorization": "Bearer YOUR_API_KEY"}  
response = requests.get(url, headers=headers)
```

```
# Example with a parameter  
params = {"api_key": "YOUR_API_KEY"}  
response = requests.get(url, params=params)
```

Table of Contents



- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)

- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- 43 Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)
- 57

2026-01-06

- Getting Data
 - Data Wrangling with Pandas
 - Table of Contents

Table of Contents

- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)

The Pandas DataFrame and Series



- ▶ **Series:** A one-dimensional labeled array, like a single column of a spreadsheet.
- ▶ **DataFrame:** A two-dimensional labeled data structure with columns of potentially different types, like a full spreadsheet or SQL table. It is the primary pandas data structure.

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

44 Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

└ Data Wrangling with Pandas

└ The Pandas DataFrame and Series

The Pandas DataFrame and Series

- ▶ **Series:** A one-dimensional labeled array, like a single column of a spreadsheet.
- ▶ **DataFrame:** A two-dimensional labeled data structure with columns of potentially different types, like a full spreadsheet or SQL table. It is the primary pandas data structure.

Get a copy of the pandas cheat sheet



You'll want this:

https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

45 Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Data Wrangling with Pandas

└─ Get a copy of the pandas cheat sheet

Get a copy of the pandas cheat sheet

You'll want this:
https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Table of Contents



- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
- Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

46

Merging and Joining DataFrames

57

Grouping and Aggregation
Reshaping Data (Pivoting)

2026-01-06

- Getting Data
 - Merging and Joining DataFrames
- Table of Contents

Table of Contents

- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
 - Grouping and Aggregation
 - Reshaping Data (Pivoting)

pd.merge() (the on, left_on, right_on parameters)



The most flexible way to combine DataFrames, similar to SQL joins.

```
df_merged = pd.merge(left_df, right_df, \
                      on='common_column', how='inner')

# how can be 'left', 'right', 'inner', 'outer'
# Use left_on and right_on if key columns have different names
```

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

47 Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Merging and Joining DataFrames

└─ pd.merge() (the on, left_on, right_on parameters)

pd.merge() (the on, left_on, right_on parameters)

The most flexible way to combine DataFrames, similar to SQL joins.

```
df_merged = pd.merge(left_df, right_df, \
                      on='common_column', how='inner')
# how can be 'left', 'right', 'inner', 'outer'
# Use left_on and right_on if key columns have different names
```

df.join() (index-based)

A convenience method for merging on indices.

```
# Joins right_df to left_df on left_df's index
df_joined = left_df.join(right_df, how='left')
```



Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

48 Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Merging and Joining DataFrames

└─ df.join() (index-based)

df.join() (index-based)

A convenience method for merging on indices.

```
# Joins right_df to left_df on left_df's index
df_joined = left_df.join(right_df, how='left')
```

pd.concat() (stacking data)



Stacks DataFrames along an axis (usually vertically, to append rows).

```
df_combined = pd.concat([df1, df2, df3]) # Stacks vertically
```

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

49 Merging and
Joining
DataFrames

Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Merging and Joining DataFrames

└─ pd.concat() (stacking data)

pd.concat() (stacking data)

Stacks DataFrames along an axis (usually vertically, to append rows).

```
df_combined = pd.concat([df1, df2, df3]) # Stacks vertically
```

Table of Contents



- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
- 50 Grouping and Aggregation
- 57 Reshaping Data (Pivoting)

2026-01-06

- Getting Data
 - Grouping and Aggregation
- Table of Contents

Table of Contents

- Data Acquisition
 - Acquiring Data from SQL
 - Combining Data from Multiple Tables
 - Acquiring Data from Web APIs
 - Example: Calling a public API
- Data Wrangling with Pandas
 - Merging and Joining DataFrames
- Grouping and Aggregation
- Reshaping Data (Pivoting)

The split-apply-combine pattern



A powerful pattern for data analysis:

1. **Split:** Break the data into groups based on some criteria (e.g., by category).

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

51 Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└ Grouping and Aggregation

└ The split-apply-combine pattern

The split-apply-combine pattern

A powerful pattern for data analysis:

1. **Split:** Break the data into groups based on some criteria (e.g., by category).

The split-apply-combine pattern



A powerful pattern for data analysis:

1. **Split:** Break the data into groups based on some criteria (e.g., by category).
2. **Apply:** Apply a function to each group independently (e.g., calculate the mean).

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

51 Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Grouping and Aggregation

└─ The split-apply-combine pattern

The split-apply-combine pattern

A powerful pattern for data analysis:

1. **Split:** Break the data into groups based on some criteria (e.g., by category).
2. **Apply:** Apply a function to each group independently (e.g., calculate the mean).

The split-apply-combine pattern



A powerful pattern for data analysis:

1. **Split:** Break the data into groups based on some criteria (e.g., by category).
2. **Apply:** Apply a function to each group independently (e.g., calculate the mean).
3. **Combine:** Combine the results into a new data structure.

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

51 Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Grouping and Aggregation

└─ The split-apply-combine pattern

The split-apply-combine pattern

A powerful pattern for data analysis:

1. **Split:** Break the data into groups based on some criteria (e.g., by category).
2. **Apply:** Apply a function to each group independently (e.g., calculate the mean).
3. **Combine:** Combine the results into a new data structure.

split-apply-combine example (code)

```
data = {
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'IT', 'IT']
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank']
    'Salary': [50000, 52000, 45000, 48000, 60000, 62000]
}
df = pd.DataFrame(data)
grouped = df.groupby('Department')
avg_salary = grouped['Salary'].mean()
df['Dept_Mean'] = grouped['Salary'].transform('mean')
high_paid_groups = \
    grouped.filter(lambda x: x['Salary'].mean() > 50000)
print(high_paid_groups)
```



Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

52 Grouping and
Aggregation

Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Grouping and Aggregation

└─ split-apply-combine example (code)

split-apply-combine example (code)

```
data = {
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'IT', 'IT']
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank']
    'Salary': [50000, 52000, 45000, 48000, 60000, 62000]
}
df = pd.DataFrame(data)
grouped = df.groupby('Department')
avg_salary = grouped['Salary'].mean()
df['Dept_Mean'] = grouped['Salary'].transform('mean')
high_paid_groups = \
    grouped.filter(lambda x: x['Salary'].mean() > 50000)
print(high_paid_groups)
```

split-apply-combine example (results)



	Department	Employee	Salary	Dept_Mean
0	Sales	Alice	50000	52333.333333
1	Sales	Bob	52000	52333.333333
4	IT	Eve	60000	60000.000000
5	IT	Frank	62000	60000.000000
6	Sales	Grace	55000	52333.333333
7	IT	Hank	58000	60000.000000

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

53 Grouping and Aggregation

Reshaping Data (Pivoting)

57

2026-01-06

Getting Data

Grouping and Aggregation

split-apply-combine example (results)

split-apply-combine example (results)

	Department	Employee	Salary	Dept_Mean
0	Sales	Alice	50000	52333.333333
1	Sales	Bob	52000	52333.333333
4	IT	Eve	60000	60000.000000
5	IT	Frank	62000	60000.000000
6	Sales	Grace	55000	52333.333333
7	IT	Hank	58000	60000.000000

Table of Contents



Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

Reshaping Data (Pivoting)

Data Acquisition

Acquiring Data from SQL

Combining Data from Multiple Tables

Acquiring Data from Web APIs

Example: Calling a public API

Data Wrangling with Pandas

Merging and Joining DataFrames

Grouping and Aggregation

54 Reshaping Data (Pivoting)

57

2026-01-06

Getting Data
└─ Reshaping Data (Pivoting)

└─ Table of Contents

Table of Contents
Data Acquisition
Acquiring Data from SQL
Combining Data from Multiple Tables
Acquiring Data from Web APIs
Example: Calling a public API
Data Wrangling with Pandas
Merging and Joining DataFrames
Grouping and Aggregation
Reshaping Data (Pivoting)

Long vs. Wide Data



- **Wide Data:** Each observation has its own row, and each variable its own column. This is the “standard” format.

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

55 Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└ Reshaping Data (Pivoting)

└ Long vs. Wide Data

Long vs. Wide Data

► **Wide Data:** Each observation has its own row, and each variable its own column. This is the “standard” format.

Long vs. Wide Data



- **Wide Data:** Each observation has its own row, and each variable its own column. This is the “standard” format.
- **Long Data (Tidy):** Each row is a single observation of a single variable. There are columns for the variable name and its value. This format is often better for analysis and plotting.

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

55 Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└─ Reshaping Data (Pivoting)

└─ Long vs. Wide Data

Long vs. Wide Data

- **Wide Data:** Each observation has its own row, and each variable its own column. This is the “standard” format.
- **Long Data (Tidy):** Each row is a single observation of a single variable. There are columns for the variable name and its value. This format is often better for analysis and plotting.

```
.pivot_table()
```

Creates a spreadsheet-style pivot table. Excellent for summarizing data from a “long” format into a “wide” format.



Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

56 Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data

└ Reshaping Data (Pivoting)

└ .pivot_table()

.pivot_table()

Creates a spreadsheet-style pivot table. Excellent for summarizing data from a “long” format into a “wide” format.

.stack() and .unstack()



- ▶ `.stack()`: Pivots a level of the column labels to the row index, making the DataFrame taller (longer).
- ▶ `.unstack()`: Pivots a level of the row index to the column labels, making the DataFrame wider.

Data Acquisition

Acquiring Data
from SQL

Combining Data
from Multiple
Tables

Acquiring Data
from Web APIs

Example: Calling
a public API

Data Wrangling
with Pandas

Merging and
Joining
DataFrames

Grouping and
Aggregation

57 Reshaping Data
(Pivoting)

57

2026-01-06

Getting Data
└─ Reshaping Data (Pivoting)

└─ `.stack()` and `.unstack()`

`.stack()` and `.unstack()`

- ▶ `.stack()`: Pivots a level of the column labels to the row index, making the DataFrame taller (longer).
- ▶ `.unstack()`: Pivots a level of the row index to the column labels, making the DataFrame wider.